

9. AI Audit Report

AI Usage Declaration

I use AI tools for the following tasks.

The AI tool used was **ChatGPT (GPT-5.6 Luna)**. I used it to help analyze Playwright E2E test failures, understand test/application mismatches, create prompts for Codex, and evaluate whether the resulting automated tests were ready for submission.

Interaction 1 — Diagnosing Playwright **Admin Login** failure

AI Tool: ChatGPT (GPT-5.6 Luna) **Date:** August 10, 2026 **Time:** 2026-08-07 7:26:28 +07:00

Prompt

Error: expect(locator).toBeVisible() failed

Locator: getByRole('heading', { name: 'Admin Login' }) Expected: visible Timeout: 5000ms Error: element(s) not found

Call log:

- Expect "toBeVisible" with timeout 5000ms
- waiting for getByRole('heading', { name: 'Admin Login' })

33 | test('shows admin login form when not authenticated', async ({ page }) => { 34 | await
page.goto('/'); 35 | await expect(page.getByRole('heading', { name: 'Admin Login' })).toBeVisible();

Why have this error, does playwright have any cache to run old test, or it run existing files (may contain new code)?

AI Output Summary

ChatGPT explained that Playwright normally does not cache old test code in the way suspected. The failure means that after navigating to `/`, Playwright could not find a visible heading with the accessible name **Admin Login**.

It suggested checking:

- whether the frontend was running;
- the Playwright **baseURL**;
- whether `/` was the correct route;
- whether the actual page contained a different heading;
- whether the application and test were targeting different frontend/backend applications;
- whether duplicate **.js** and **.ts** test files existed;
- running the test with `--headed` or `--debug`;
- taking screenshots to inspect what Playwright actually rendered.

The AI also noted that the generated test might be based on assumptions about a UI that did not actually exist.

Interaction 2 — Prompt to make Codex scan the application before fixing tests

AI Tool: ChatGPT (GPT-5.6 Luna) **Date:** August 10, 2026 **Time:** 2026-08-07 7:37:43 +07:00

Prompt

prompt for codex to fix tests in folder e2e-tests, there are many "element not found" error, let codex scan code in folder apps first

AI Output

ChatGPT provided a Codex prompt instructing Codex to:

1. Inspect the `apps` folder first.
2. Understand the frontend/backend structure.
3. Identify actual routes, UI components, labels, authentication flow, and API endpoints.
4. Inspect every Playwright test in `e2e-tests`.
5. Compare test assumptions with the actual application.
6. Fix incorrect routes, selectors, forms, authentication setup, and expected messages.
7. Prefer stable Playwright selectors such as `getByRole`, `getByLabel`, and `getByPlaceholder`.
8. Avoid inventing UI elements.
9. Run `npx playwright test` after fixing.
10. Report modified files, root causes, and final test results.

The purpose was to make Codex analyze the real application instead of blindly changing selectors.

Interaction 3 — Evaluating the report showing structural test failures

AI Tool: ChatGPT (GPT-5.6 Luna) **Date:** August 10, 2026 **Time:** 2026-08-07 7:43:21 +07:00

Prompt

The report shows the failures are mostly from test assumptions not matching the real UI:

- Some tests were opening the wrong app or route, especially hitting the backend or reloading checkout directly instead of going through the web frontend.
- Several selectors were wrong because the UI does not expose the labels the tests expected as real accessible labels.
- Checkout failed because the cart is React state, so going straight to `/checkout` clears it.
- Admin/profile tests also needed real seeded login and order data, not fake `localStorage` values.
- A few assertions expected texts like `Admin Login`, `Họ Tên`, `Số điện thoại`, or status strings that didn't line up with the current rendered UI.

In short: the failures come from route mismatches, state-reset issues, and selectors/messages that do not exist exactly as the tests assumed. I ask codex to scan src code but still mismatch, assume?

AI Output Summary

ChatGPT explained that source-code scanning alone may not be sufficient because:

- source code does not always reveal the actual rendered accessibility tree;
- frontend and backend routes can be different;
- React state can be lost when directly navigating to a page;
- authentication may depend on real backend state rather than arbitrary localStorage values;
- AI-generated tests can describe an imagined UI rather than the real user journey.

It recommended that Codex validate the running application with Playwright, inspect the DOM/accessibility tree, reproduce actual workflows, and then modify tests.

Interaction 4 — Evaluating whether the tests were good enough

AI Tool: ChatGPT (GPT-5.6 Luna) **Date:** August 10, 2026 **Time:** 2026-08-07 7:56:43 +07:00

Prompt

The live app confirms the tests fail for structural reasons, not just brittle selectors:

- <http://localhost:5173/profile> shows **Vui lòng đăng nhập** unless the app's real auth state is established; the profile form is not reliably available from a direct visit.
- <http://localhost:5173/checkout> is not a good direct entry point for cart tests, because checkout depends on React cart state built by adding items from the home page first.
- <http://localhost:5174/> is the admin dashboard, not an admin login page after auth. The visible text is **EShop Admin, Dashboard, Đơn hàng**, etc.
- Some expected labels in tests are not actually accessible labels in the DOM, so `getByLabel()` can fail even when the text is visible on screen.
- The tests are still using the wrong user flow in places, especially direct navigation instead of login/add-to-cart/navigation through the UI.

In short: the failures come from using the wrong flow, not matching the real visible text/accessibility tree, and assuming pages can be opened directly when they actually need prior React state or auth. Are the tests good enough?

AI Output Summary

ChatGPT concluded that the tests were not yet good enough as final submission.

It explained that the tests had the correct general testing intent but did not represent realistic E2E user behavior.

Examples given:

Profile:

```
Login
→ Profile
→ Edit profile
→ Save
→ Verify
```

instead of directly opening `/profile`.

Checkout:

```
Home
→ Select product
→ Add to cart
→ Cart
→ Checkout
```

instead of directly opening `/checkout`.

Admin:

```
Admin authentication
→ Dashboard
→ Orders
→ Change order status
```

instead of assuming an `Admin Login` page existed.

The AI rated the current tests approximately:

- Test idea: 7/10
- Test implementation: 4/10
- Real user flow: 3/10
- Selector quality: 5/10
- Ready for submission: No

Interaction 5 — Codex prompt to rewrite tests using actual user flows

AI Tool: ChatGPT (GPT-5.6 Luna) **Date:** August 10, 2026 **Time:** 2026-08-08 15:32:35 +07:00

Prompt

```
prompt to fix
```

AI Output

ChatGPT generated a Codex prompt instructing Codex to rewrite the Playwright tests based on real user workflows rather than simply patching selectors.

The prompt required:

- real authentication before profile tests;
- adding products before checkout;
- following the actual admin authentication/dashboard flow;
- using real DOM/accessibility selectors;
- preserving the HW04 requirements;
- running the tests and debugging remaining failures.

Interaction 6 — Playwright command-line usage

AI Tool: ChatGPT (GPT-5.6 Luna) **Date:** August 10, 2026 **Time:** 2026-08-08 15:40:47 +07:00

Prompt

```
command line to run playwright
```

AI Output

ChatGPT provided commands including:

```
npx playwright test
```

Run one test file:

```
npx playwright test e2e-tests/admin-orders.spec.ts
```

Run with browser visible:

```
npx playwright test --headed
```

Run in debug mode:

```
npx playwright test e2e-tests/admin-orders.spec.ts --debug
```

Run individual browsers:

```
npx playwright test --project=chromium  
npx playwright test --project=firefox
```

```
npx playwright test --project=webkit
```

Open the HTML report:

```
npx playwright show-report
```

Interaction 7 — Diagnosing profile `page.evaluate()` timeout

AI Tool: ChatGPT (GPT-5.6 Luna) **Date:** August 10, 2026 **Time:** 2026-08-08 16:01:03 +07:00

Prompt

Error: page.evaluate: Test timeout of 30000ms exceeded.

```
49 | 50 | async function triggerSubmit(page) { 51 | await page.evaluate() => { 52 | const button =  
document.querySelector('button[type="submit"]') as HTMLButtonElement | null; 53 | button?.click(); 54 |  
}); 55 | }
```

... is the test good enough?

AI Output Summary

ChatGPT explained that the test was still not good enough.

It identified the following pattern as problematic:

```
await page.evaluate(() => {  
  const button = document.querySelector('button[type="submit"]');  
  button?.click();  
});
```

The AI recommended normal Playwright interactions such as:

```
await page.locator('button[type="submit"]').click();
```

or preferably an accessible role selector:

```
await page.getByRole('button', { name: /save|update/i }).click();
```

It also explained that optional chaining (`button?.click()`) can silently do nothing if the button is missing, making the test less reliable.

The AI recommended following the complete profile user flow and verifying the actual post-submit result.

Interaction 8 — Prompt to fix profile test

AI Tool: ChatGPT (GPT-5.6 Luna) **Date:** August 10, 2026 **Time:** 2026-08-08 16:17:57 +07:00

Prompt

prompt to fix

AI Output

ChatGPT generated a Codex prompt specifically for `profile.spec.ts`.

The prompt instructed Codex to:

- establish real authentication;
- navigate to the profile page;
- use actual profile fields;
- avoid `page.evaluate()`;
- avoid `button?.click()`;
- use Playwright-native interactions;
- wait for the actual API response, redirect, or visible success/error state;
- verify that the profile update actually succeeded;
- run the profile test after fixing.

Interaction 9 — Diagnosing `Cập nhật` timeout

AI Tool: ChatGPT (GPT-5.6 Luna) **Date:** August 10, 2026 **Time:** 2026-08-09 19:28:36 +07:00

Prompt

updates profile with valid data: Nguyen Van A e2e-tests/profile.spec.ts:84 35.0s Chromium

Error: locator.click: Test timeout of 30000ms exceeded.

Call log:

- waiting for getByRole('button', { name: 'Cập nhật' })
- locator resolved to Cập nhật
- attempting click action
- waiting for element to be visible, enabled and stable
- element is visible, enabled and stable
- scrolling into view if needed
- done scrolling
- performing click action

... is the test good?

AI Output Summary

ChatGPT determined that the selector itself was now working because Playwright successfully resolved the **Cập nhật** button and performed the click action.

The likely problem was the code:

```
await Promise.all([
  dialogPromise,
  page.getByRole('button', { name: 'Cập nhật' }).click(),
]);
```

The AI explained that if the application does not actually produce a browser dialog, a `page.waitForEvent('dialog')` promise can wait until the entire 30-second test timeout.

It recommended checking the actual post-submit behavior and asserting the real in-page message, DOM change, API response, or redirect instead of waiting for a nonexistent dialog.

Interaction 10 — Whether to submit with 105 passed / 21 failed

AI Tool: ChatGPT (GPT-5.6 Luna) **Date:** August 10, 2026 **Time:** 2026-08-09 19:39:23 +07:00

Prompt

is it oke to submit right now, there are more tests, should I check each manually, 105 passed , 21 failed, is it oke to submit to homework, example test results:

Error: locator.click: Test timeout of 30000ms exceeded.

```
... getByRole('button', { name: 'Cập nhật' })
```

```
... Promise.all([dialogPromise, ...])
```

```
... profile.spec.ts
```

AI Output Summary

ChatGPT advised **not to submit yet** with 105 passed and 21 failed.

It calculated that approximately 83.3% of tests passed and 16.7% failed.

However, it also advised against manually inspecting all 126 tests individually.

Instead, the recommendation was to:

1. Read the Playwright report.
2. Group the 21 failures by root cause.
3. Fix shared helper problems first.
4. Rerun affected tests.
5. Investigate the remaining failures individually.
6. Distinguish genuine application defects from test defects.

The AI specifically noted that the profile `dialogPromise` problem could cause multiple tests to fail because they share the same `updateProfile()` helper.

The recommended target was to achieve a clean suite where possible, or document any remaining failures as genuine application defects rather than submitting unexplained test failures.

Interaction 11 — Prompt to have Codex read the Playwright report and fix tests

AI Tool: ChatGPT (GPT-5.6 Luna) **Date:** August 10, 2026 **Time:** 2026-08-09 19:51:29 +07:00

Prompt

in folder HW04, prompt for codex to read playwright report then fix tests like that in folder e2e-tests

AI Output

ChatGPT provided a detailed Codex prompt requiring Codex to:

1. Read the existing Playwright HTML report and test-results artifacts first.
2. Identify and group the 21 failures by root cause.
3. Inspect the real application under `apps/`.
4. Compare test assumptions against actual application behavior.
5. Correct profile authentication and navigation.
6. Correct checkout by creating cart state through UI actions.
7. Correct admin authentication and dashboard flow.
8. Use real DOM/accessibility selectors.
9. Remove `page.evaluate()` for normal UI interaction.
10. Remove fake authentication.
11. Avoid increasing timeouts to hide failures.
12. Preserve HW04 requirements.
13. Run the complete Playwright suite again.
14. Provide a final summary of modified files, root causes, fixes, and test results.

Overall AI Contribution

AI was used as an assistant for:

- interpreting Playwright error messages;
- diagnosing incorrect E2E test assumptions;
- identifying route and authentication problems;
- identifying React state problems in checkout;
- identifying selector/accessibility issues;
- generating prompts for Codex to inspect and modify the test suite;
- suggesting Playwright commands for test execution and debugging;
- evaluating test quality and readiness for submission;
- suggesting a systematic approach to grouping and fixing test failures.

The AI did not replace the execution and verification of the tests. The tests were run against the actual application, and the observed Playwright reports and failures were used to guide subsequent debugging.